

LAPORAN PRATIKUM STRUKTUR DATA

9



Dosen Pengampu:

DR. Wahyudi. .S.T.M.T

Asisten Lab:

Jovantri Immanuel Gulo

Disusun Oleh:

Marcello Bayu Denar Widjaksono

2511532011

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer, struktur data non-linear seperti Pohon Biner (Binary Tree) dan Graf (Graph) memegang peranan krusial dalam memodelkan hubungan hierarkis dan jaringan yang kompleks. Berbeda dengan struktur data linear (seperti array atau linked list), pohon dan graf memerlukan pendekatan manajemen memori dan penelusuran yang lebih spesifik, seperti penggunaan pointer (referensi objek) dan algoritma rekursi. Pemahaman mendalam mengenai implementasi node, pengaturan anak kiri dan kanan pada pohon, serta representasi Adjacency List pada graf, menjadi fondasi penting dalam pengembangan perangkat lunak. Selain itu, penguasaan terhadap algoritma penelusuran (traversal) seperti Preorder, Inorder, Postorder, Depth-First Search (DFS), dan Breadth-First Search (BFS) sangat diperlukan untuk menyelesaikan berbagai masalah komputasi, mulai dari pengurutan data, pencarian jalur terpendek, hingga analisis jaringan. Oleh karena itu, praktikum ini dilakukan untuk mengimplementasikan konsep-konsep tersebut secara langsung menggunakan bahasa pemrograman Java.

1.2 Tujuan Pratikum

1. Bagaimana cara mengimplementasikan struktur data Pohon Biner dan Graf beserta operasi dasarnya (seperti pencarian dan penghitungan simpul) dalam bahasa Java?
2. Bagaimana mekanisme kerja algoritma penelusuran (traversal) Preorder, Inorder, Postorder, DFS, dan BFS dalam menavigasi struktur data non-linear secara efisien?

1.3 Manfaat Pratikum

1. Meningkatkan pemahaman konseptual dan keterampilan pemrograman dalam mengelola struktur data non-linear, termasuk penerapan logika rekursi dan penggunaan struktur data pendukung (seperti Queue dan HashSet) untuk mencegah infinite loop dan duplikasi data.

2. Memberikan landasan yang kuat dalam pengembangan solusi perangkat lunak nyata yang memerlukan pemodelan hierarki (misalnya: struktur direktori file, decision tree) atau pemetaan jaringan hubungan (misalnya: rute navigasi, jejaring sosial, dan topologi jaringan)..

BAB II PEMBAHASAN

2.1 Pembahasan Pratikum

```
package pekan9_2511532011;

public class Node_2511532011 {
    int data_2011;
    Node_2511532011 left_2011;
    Node_2511532011 right_2011;

    public Node_2511532011(int data_2011) {
        this.data_2011 = data_2011;
        left_2011 = null;
        right_2011 = null;
    }

    public void setLeft_2011(Node_2511532011 node_2011) {
        if ( left_2011 == null ) {
            left_2011 = node_2011;
        }
    }

    public void setRight_2011(Node_2511532011 node_2011) {
        if (right_2011 == null ) {
            right_2011 = node_2011;
        }
    }

    public Node_2511532011 getLeft_2011() {
        return left_2011;
    }

    public Node_2511532011 getRight_2011() {
        return right_2011;
    }

    public int getData_2011() {
        return data_2011;
    }

    public void setData_2011(int data_2011) {
        this.data_2011 = data_2011;
    }

    void printPreorder_2011(Node_2511532011 node_2011) {
        if(node_2011 == null ) {
            return;
        }
        System.out.print(node_2011.data_2011 + " ");
        printPreorder_2011(node_2011.left_2011);
        printPreorder_2011(node_2011.right_2011);
    }

    void printPostorder_2011(Node_2511532011 node_2011) {
        if(node_2011 == null ) {
            return;
        }
        printPostorder_2011(node_2011.left_2011);
        printPostorder_2011(node_2011.right_2011);
        System.out.print(node_2011.data_2011 + " ");
    }

    void printInorder_2011(Node_2511532011 node_2011) {
        if(node_2011 == null ) {
            return;
        }
        printInorder_2011(node_2011.left_2011);
        System.out.print(node_2011.data_2011 + " ");
        printInorder_2011(node_2011.right_2011);
    }

    public String print_2011() {
        return this.print_2011("", true, "");
    }

    public String print_2011(String prefix_2011, boolean isTail_2011, String sb_2011) {
        if (right_2011 != null ) {
            right_2011.print_2011(prefix_2011 + (isTail_2011 ? "| " : " "), false, sb_2011);
        }
        System.out.println( prefix_2011 + (isTail_2011 ? "\\-- " : "/-- ") + data_2011);
        if (left_2011 != null) {
            left_2011.print_2011(prefix_2011 + (isTail_2011 ? " " : "| "), true, sb_2011);
        }
        return sb_2011;
    }
}
```

Pembahasan Class Node_2511532011:

1. Deklarasi Kelas

`public class Node_2511532011:` Mendefinisikan kelas publik sebagai komponen fundamental dalam struktur data pohon biner (Binary Tree).

Kelas ini bersifat self-referential karena memiliki atribut yang bertipe sama dengan kelas itu sendiri (`left_2011`, `right_2011`).

2. Atribut Data

`int data_2011;` Variabel instance bertipe integer untuk menyimpan nilai atau payload yang dibawa oleh node ini.

Atribut Pointer Anak Kiri

`Node_2511532011 left_2011;` Referensi ke simpul anak sebelah kiri. Pada pohon biner, setiap node maksimal memiliki satu anak kiri.

3. Atribut Pointer Anak Kanan

`Node_2511532011 right_2011;` Referensi ke simpul anak sebelah kanan.

4. Konstruktor Node

`public Node_2511532011(int data_2011):` Konstruktor yang menerima nilai data untuk inisialisasi node baru.

`this.data_2011 = data_2011;` Assign nilai parameter ke atribut instance.

`left_2011 = null; right_2011 = null;` Inisialisasi pointer anak dengan null, menandakan node baru ini adalah daun (leaf) dan belum memiliki cabang.

5. Metode Setter Anak Kiri

`public void setLeft_2011(Node_2511532011 node_2011):` Menetapkan referensi node sebagai anak kiri.

`if (left_2011 == null) left_2011 = node_2011;` Proteksi Overwrite. Hanya mengisi jika slot kiri masih kosong, mencegah penimpaan subtree yang sudah ada.

6. Metode Setter Anak Kanan

`public void setRight_2011(Node_2511532011 node_2011):` Sama seperti setter kiri, namun untuk slot kanan. Menjamin integritas struktur biner (maksimal 2 anak per node).

7. Metode Getter

getLeft_2011(), getRight_2011(), getData_2011(): Metode publik untuk mengakses atribut privat/package-private dari luar kelas, menerapkan prinsip enkapsulasi dasar.

setData_2011(int data_2011): Memungkinkan perubahan nilai data node setelah inisialisasi.

8. Metode Traversal Preorder

void printPreorder_2011(Node_2511532011 node_2011): Mencetak node dengan pola Root → Left → Right.

if (node_2011 == null) return;; Base case rekursi. Menghentikan eksekusi jika mencapai leaf.

System.out.print(node_2011.data_2011 + " ");: Cetak data root terlebih dahulu.

printPreorder_2011(node_2011.left_2011);: Rekursif ke subtree kiri.

printPreorder_2011(node_2011.right_2011);: Rekursif ke subtree kanan.

Use Case: Berguna untuk menyalin/mengkopi struktur pohon atau membuat prefix expression.

9. Metode Traversal Postorder

void printPostorder_2011(Node_2511532011 node_2011): Pola Left → Right → Root.

Rekursi kiri dan kanan dipanggil terlebih dahulu, baru mencetak data root.

Use Case: Berguna untuk menghapus node secara aman (hapus anak dulu, baru parent) atau menghasilkan postfix expression.

10. Metode Traversal Inorder

void printInorder_2011(Node_2511532011 node_2011): Pola Left → Root → Right.

Use Case: Jika pohon ini adalah Binary Search Tree (BST), inorder traversal akan menghasilkan urutan data yang terurut secara menaik (ascending).


```

package pekan9_2511532011;

public class Btree_2511532011 {
    private Node_2511532011 root_2011;
    private Node_2511532011 currentNode_2011;

    public Btree_2511532011() {
        root_2011 = null;
    }

    public boolean search_2011(int data_2011) {
        return search_2011(root_2011, data_2011);
    }

    private boolean search_2011(Node_2511532011 node_2011, int data_2011) {
        if (node_2011.getData_2011() == data_2011)
            return true;
        if (node_2011.getLeft_2011() != null)
            if (search_2011(node_2011.getLeft_2011(), data_2011))
                return true;
        if (node_2011.getRight_2011() != null)
            if (search_2011(node_2011.getRight_2011(), data_2011))
                return true;
        return false;
    }

    public void printInorder_2011() {
        root_2011.printInorder_2011(root_2011);
    }

    public void printPreorder_2011() {
        root_2011.printPreorder_2011(root_2011);
    }
    public void printPostorder_2011() {
        root_2011.printPostorder_2011(root_2011);
    }

    public Node_2511532011 getRoot_2011() {
        return root_2011;
    }

    public boolean isEmpty_2011() {
        return root_2011 == null;
    }

    public int countNodes_2011() {
        return countNodes_2011(root_2011);
    }

    private int countNodes_2011(Node_2511532011 node_2011) {
        int count_2011 = 1;
        if (node_2011 == null) {
            return 0;
        } else {
            count_2011 += countNodes_2011(node_2011.getLeft_2011());
            count_2011 += countNodes_2011(node_2011.getRight_2011());
            return count_2011;
        }
    }

    public void print_2011() {
        root_2011.print_2011();
    }

    public Node_2511532011 getCurrent_2011() {
        return currentNode_2011;
    }

    public void setCurrent_2011(Node_2511532011 node_2011) {
        this.currentNode_2011 = node_2011;
    }

    public void setRoot_2011(Node_2511532011 root_2011) {
        this.root_2011 = root_2011;
    }
}

```

Pembahasan class Btree_2511532011:

1. Deklarasi Kelas dan Atribut

public class BTree_2511532011: Kelas publik yang mengabstraksi operasi tingkat pohon, menyembunyikan kompleksitas manipulasi node individual dari pengguna akhir.

private Node_2511532011 root_2011;; Referensi ke simpul akar (root). Merupakan titik masuk utama untuk seluruh operasi pada pohon.

private Node_2511532011 currentNode_2011;; Pointer atau "kursor" yang menunjuk ke node tertentu. Pola ini umum digunakan untuk navigasi

interaktif, operasi insert/delete bertahap, atau integrasi dengan antarmuka GUI.

2. Konstruktor

`public BTree_2511532011() { root_2011 = null; }`: Menginisialisasi pohon dalam keadaan kosong. `currentNode_2011` secara otomatis bernilai null karena tidak diinisialisasi secara eksplisit.

3. Metode `search_2011` (Pencarian)

Wrapper Publik: `public boolean search_2011(int data_2011) →` Memanggil metode rekursif privat dengan parameter `root_2011`.

Implementasi Rekursif Privat: `private boolean search_2011(Node_2511532011 node_2011, int data_2011)`

`if (node_2011.getData_2011() == data_2011) return true;` Mengecek apakah node saat ini menyimpan data target.

`if (node_2011.getLeft_2011() != null) if (search_2011(...)) return true;` Jika belum ketemu, lakukan pencarian DFS (Depth-First Search) ke subtree kiri.

`if (node_2011.getRight_2011() != null) if (search_2011(...)) return true;` Jika di kiri tidak ada, lanjutkan ke subtree kanan.

`return false;` Dikembalikan jika seluruh cabang telah dijelajahi tanpa menemukan data.

Catatan Kritis: Metode ini tidak menangani kasus `node_2011 == null` di awal pemanggilan rekursif. Jika pohon kosong (`root_2011 null`), pemanggilan akan menghasilkan `NullPointerException`. Sebaiknya ditambahkan guard clause `if (node_2011 == null) return false;` di baris pertama.

4. Metode Wrapper Traversal

`printInorder_2011()`, `printPreorder_2011()`, `printPostorder_2011()`: Ketiga metode ini hanya berfungsi sebagai delegator. Mereka memanggil metode traversal yang sesuai pada objek `root_2011`.

Pola ini menerapkan prinsip Façade Pattern, menyederhanakan antarmuka sehingga pengguna `BTree` tidak perlu mengakses metode internal kelas `Node` secara langsung.

5. Metode Utilitas dan Penghitungan

`getRoot_2011()`, `isEmpty_2011()`: Akses dasar untuk memeriksa state pohon.

`countNodes_2011()` (Publik & Privat): Menghitung total simpul secara rekursif.

Logika: `if (null) return 0` → basis kasus. `else` → 1 (node sendiri) + hitung kiri + hitung kanan.

Catatan Kode: Baris `int count_2011 = 1;` dideklarasikan sebelum pengecekan null. Meskipun tidak menyebabkan error karena langsung di-skip oleh `return 0`, ini adalah dead code yang kurang bersih. Inisialisasi seharusnya berada di dalam blok `else`.

`print_2011()`: Mendelegasikan pemanggilan ke metode visualisasi ASCII tree pada `root_2011`.

6. Manajemen State (Current Node)

`getCurrent_2011()`, `setCurrent_2011()`, `setRoot_2011()`: Metode getter/setter untuk pointer `currentNode` dan `root`.

Keberadaan `currentNode` mengindikasikan bahwa kelas ini dirancang untuk mendukung operasi cursor-based manipulation, di mana pengguna dapat memindahkan fokus ke node tertentu sebelum melakukan operasi sisip/hapus (meskipun metode tersebut belum diimplementasikan di kelas ini).

```

public class BtreeDriver_2511532011 {

    public static void main(String[] args) {

        Btree_2511532011 tree_2011 = new Btree_2511532011();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_2011.countNodes_2011());

        Node_2511532011 root_2011 = new Node_2511532011(1);

        tree_2011.setRoot_2011(root_2011);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_2011.countNodes_2011());
        Node_2511532011 node2_2011 = new Node_2511532011(2);
        Node_2511532011 node3_2011 = new Node_2511532011(3);
        Node_2511532011 node4_2011 = new Node_2511532011(4);
        Node_2511532011 node5_2011 = new Node_2511532011(5);
        Node_2511532011 node6_2011 = new Node_2511532011(6);
        Node_2511532011 node7_2011 = new Node_2511532011(7);
        Node_2511532011 node8_2011 = new Node_2511532011(8);
        Node_2511532011 node9_2011 = new Node_2511532011(9);
        root_2011.setLeft_2011(node2_2011);
        node2_2011.setLeft_2011(node4_2011);
        node2_2011.setRight_2011(node5_2011);
        node4_2011.setRight_2011(node8_2011);
        root_2011.setRight_2011(node3_2011);
        node3_2011.setLeft_2011(node6_2011);
        node3_2011.setRight_2011(node7_2011);
        node6_2011.setLeft_2011(node9_2011);

        tree_2011.setCurrent_2011(tree_2011.getRoot_2011());
        System.out.println("menampilkan simpul terakhir: ");
        System.out.println(tree_2011.getCurrent_2011().getData_2011());
        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
        System.out.println(tree_2011.countNodes_2011());
        System.out.println("InOrder: ");
        tree_2011.printInorder_2011();
        System.out.println("\nPreorder: ");
        tree_2011.printPreorder_2011();
        System.out.println("\nPostorder: ");
        tree_2011.printPostorder_2011();
        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
        tree_2011.print_2011();

    }

}

```

Pembahasan Class BtreeDriver_2511532011:

1. Deklarasi Kelas & Metode Utama

public class BTree_2511532011: Kelas driver yang tidak menyimpan logika pohon, melainkan bertugas menguji dan mendemonstrasikan penggunaan API yang disediakan oleh kelas BTree dan Node.

public static void main(String[] args): Titik masuk program dimana instansiasi, konstruksi, dan pengujian dilakukan.

2. Inisialisasi Objek Pohon

BTree_2511532011 tree_2011 = new BTree_2511532011();: Membuat instance pohon biner. Secara internal, konstruktor BTree mengatur root_2011 = null, menandakan pohon dalam keadaan kosong.

tree_2011.countNodes_2011(): Memanggil metode penghitung pada pohon kosong. Berdasarkan implementasi sebelumnya, jika root null, metode mengembalikan 0.

3. Penentuan Simpul Akar (Root)

Node_2511532011 root_2011 = new Node_2511532011(1);: Membuat node pertama dengan nilai 1.

tree_2011.setRoot_2011(root_2011);: Mendaftarkan node ini sebagai akar pohon melalui metode setter.

Pemanggilan countNodes_2011() kembali menghasilkan 1, memverifikasi bahwa root telah terikat dengan benar.

4. Konstruksi Manual Struktur Pohon

Kode membuat 8 node tambahan (node2 sampai node9) dengan nilai 2 hingga 9.

Penghubungan dilakukan secara eksplisit menggunakan metode setLeft_2011() dan setRight_2011(). Ini adalah teknik umum dalam pengujian struktur data sebelum metode insert() otomatis diimplementasikan.

Peta Koneksi:

1 → kiri: 2, kanan: 3

2 → kiri: 4, kanan: 5

4 → kanan: 8

3 → kiri: 6, kanan: 7

6 → kiri: 9

5. Manajemen Pointer currentNode

tree_2011.setCurrent_2011(tree_2011.getRoot_2011());: Mengatur pointer navigasi (currentNode) agar menunjuk ke root. Pola ini berguna untuk operasi berbasis kursor atau integrasi GUI.

`tree_2011.getCurrent_2011().getData_2011()`: Mengakses dan mencetak data node yang sedang aktif (output: 1).

6. Verifikasi Total Simpul

`tree_2011.countNodes_2011()`: Dipanggil setelah semua koneksi terbentuk. Metode rekursif akan menjelajahi seluruh cabang dan mengembalikan total 9.

Catatan: Komentar kode menyebut "setelah simpul 7 ditambahkan", namun secara faktual seluruh node 2-9 telah terhubung. Output tetap 9.

7. Eksekusi Traversal Inorder

`tree_2011.printInorder_2011()`: Memanggil delegator yang menjalankan pola Left → Root → Right mulai dari root.

Alur: (8 4) 2 5 → 1 → (9 6) 3 7

Output: 4 8 2 5 1 9 6 3 7

8. Eksekusi Traversal Preorder

`tree_2011.printPreorder_2011()`: Pola Root → Left → Right.

Alur: 1 → (2 (4 8) 5) → (3 (6 9) 7)

Output: 1 2 4 8 5 3 6 9 7

9. Eksekusi Traversal Postorder

`tree_2011.printPostorder_2011()`: Pola Left → Right → Root.

Alur: (8 4) 5 2 → (9 6) 7 3 → 1

Output: 8 4 5 2 9 6 7 3 1

```

package pekan9_2511532011;

import java.util.Map;
import java.util.List;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.Set;
import java.util.Queue;

public class GraphTraversal_2511532011 {
    private Map<String, List<String>> graph_2011 = new HashMap<>();

    public void addEdge_2011(String node1_2011, String node2_2011) {
        graph_2011.putIfAbsent(node1_2011, new ArrayList<>());
        graph_2011.putIfAbsent(node2_2011, new ArrayList<>());
        graph_2011.get(node1_2011).add(node2_2011);
        graph_2011.get(node2_2011).add(node1_2011);
    }

    public void printGraph_2011() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_2011 : graph_2011.keySet()) {
            System.out.print(node_2011 + " -> ");
            List<String> neighbors_2011 = graph_2011.get(node_2011);
            System.out.println(String.join(", ", neighbors_2011));
        }
        System.out.println();
    }

    public void dfs_2011(String start_2011) {
        Set<String> visited_2011 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper_2011(start_2011, visited_2011);
        System.out.println();
    }

    private void dfsHelper_2011(String current_2011, Set<String> visited_2011) {
        if (visited_2011.contains(current_2011))
            return;
        visited_2011.add(current_2011);
        System.out.print(current_2011 + " ");
        for (String neighbor_2011 : graph_2011.getOrDefault(current_2011, new ArrayList<>())) {
            dfsHelper_2011(neighbor_2011, visited_2011);
        }
    }

    public void bfs_2011(String start_2011) {
        Set<String> visited_2011 = new HashSet<>();
        Queue<String> queue_2011 = new LinkedList<>();
        queue_2011.add(start_2011);
        visited_2011.add(start_2011);
        System.out.println("Penelusuran BFS:");
        while (!queue_2011.isEmpty()) {
            String current_2011 = queue_2011.poll();
            System.out.print(current_2011 + " ");
            for (String neighbor_2011 : graph_2011.getOrDefault(current_2011, new ArrayList<>())) {
                if (!visited_2011.contains(neighbor_2011)) {
                    queue_2011.add(neighbor_2011);
                    visited_2011.add(neighbor_2011);
                }
            }
        }
        System.out.println();
    }

    public static void main(String[] args) {
        GraphTraversal_2511532011 graph_2011 = new GraphTraversal_2511532011();

        graph_2011.addEdge_2011("A", "B");
        graph_2011.addEdge_2011("A", "C");
        graph_2011.addEdge_2011("B", "D");
        graph_2011.addEdge_2011("B", "E");

        System.out.println("Graf Awal adalah: ");
        graph_2011.printGraph_2011();

        graph_2011.dfs_2011("A");
        graph_2011.bfs_2011("A");
    }
}

```

Pembahasan Class GraphTransversal_2511532011:

1. Deklarasi Kelas dan Struktur Data Utama

`public class GraphTraversal_2511532011:` Kelas publik yang berfungsi sebagai manajer untuk operasi graf: konstruksi, visualisasi, dan traversal.

`private Map<String, List<String>> graph_2011 = new HashMap<>();`

Adjacency List. Struktur data inti yang merepresentasikan graf:

`Map<String, ...>:` Key adalah label node (String seperti "A", "B").

`List<String>:` Value adalah daftar tetangga (neighbors) dari node tersebut.

`HashMap:` Memberikan akses $O(1)$ untuk mencari daftar tetangga suatu node.

2. Metode addEdge_2011 (Konstruksi Graf Tak Berarah)

`public void addEdge_2011(String node1_2011, String node2_2011):`

Menambahkan sisi (edge) antara dua node.

`graph_2011.putIfAbsent(node1_2011, new ArrayList<>());` Memastikan node1 sudah memiliki entri dalam map. Jika belum, buat list kosong.

`graph_2011.putIfAbsent(node2_2011, new ArrayList<>());` Sama untuk node2.

`graph_2011.get(node1_2011).add(node2_2011);` Tambahkan node2 ke daftar tetangga node1.

`graph_2011.get(node2_2011).add(node1_2011);` Penting: Tambahkan node1 ke daftar tetangga node2. Ini menjadikan graf undirected (dua arah): jika A terhubung ke B, maka B juga terhubung ke A.

3. Metode printGraph_2011 (Visualisasi Adjacency List)

`for (String node_2011 : graph_2011.keySet()):` Iterasi melalui semua node dalam graf.

`List<String> neighbors_2011 = graph_2011.get(node_2011);` Ambil daftar tetangga node saat ini.

`System.out.println(String.join(", ", neighbors_2011));` Cetak tetangga dipisahkan koma menggunakan utility `String.join` untuk format yang rapi.

Output contoh: A -> B, C berarti node A terhubung langsung ke B dan C.

4. Metode dfs_2011 (Wrapper Depth-First Search)

public void dfs_2011(String start_2011): Metode publik untuk memulai traversal DFS dari node awal.

Set<String> visited_2011 = new HashSet<>();: Membuat set untuk melacak node yang sudah dikunjungi. Penting untuk mencegah infinite loop pada graf yang memiliki siklus.

dfsHelper_2011(start_2011, visited_2011);: Memanggil metode rekursif privat dengan state visited yang sama.

5. Metode dfsHelper_2011 (Inti Rekursi DFS)

private void dfsHelper_2011(String current_2011, Set<String> visited_2011): Implementasi rekursif DFS.

if (visited_2011.contains(current_2011)) return;: Base Case. Jika node sudah dikunjungi, hentikan rekursi untuk cabang ini.

visited_2011.add(current_2011);: Tandai node saat ini sebagai visited.

System.out.print(current_2011 + " ");: Proses Node. Cetak node saat ini sebelum menjelajahi anak-anaknya (pre-order).

for (String neighbor_2011 : graph_2011.getDefault(current_2011, new ArrayList<>())): Iterasi semua tetangga. getDefault mencegah NullPointerException jika node tidak memiliki tetangga.

dfsHelper_2011(neighbor_2011, visited_2011);: Rekursi. Jelajahi tetangga secara mendalam sebelum kembali (backtrack).

Karakteristik DFS: Menjelajah sedalam mungkin di satu cabang sebelum backtrack. Cocok untuk deteksi siklus, topological sort, atau mencari jalur di maze.

6. Metode bfs_2011 (Breadth-First Search Iteratif)

public void bfs_2011(String start_2011): Metode untuk traversal BFS dari node awal.

Set<String> visited_2011 = new HashSet<>();: Set untuk pelacakan node visited.

Queue<String> queue_2011 = new LinkedList<>();: Struktur Data Kunci. Queue (FIFO) memastikan node diproses berdasarkan level/kedekatan dari start node.

queue_2011.add(start_2011); visited_2011.add(start_2011);: Inisialisasi: enqueue dan tandai node awal.

while (!queue_2011.isEmpty()): Loop utama berjalan selama masih ada node dalam antrian.

String current_2011 = queue_2011.poll();: Dequeue node paling depan untuk diproses.

System.out.print(current_2011 + " ");: Cetak node saat ini.

for (String neighbor_2011 : graph_2011.getDefault(...)): Iterasi tetangga node saat ini.

if (!visited_2011.contains(neighbor_2011)): Hanya proses tetangga yang belum visited.

queue_2011.add(neighbor_2011);: Enqueue tetangga untuk diproses di level berikutnya.

visited_2011.add(neighbor_2011);: Penting: Tandai visited saat enqueue (bukan saat dequeue) untuk mencegah duplikasi node dalam queue.

Karakteristik BFS: Menjelajah level demi level (semua tetangga jarak-1, lalu jarak-2, dst). Cocok untuk mencari jalur terpendek dalam graf tak berbobot.

7. Metode main - Konstruksi Graf Contoh

GraphTraversal_2011 graph_2011 = new GraphTraversal_2011();: Instansiasi objek graf.

addEdge_2011("A", "B"); addEdge_2011("A", "C");: Node A terhubung ke B dan C.

addEdge_2011("B", "D"); addEdge_2011("B", "E");: Node B terhubung ke D dan E.

8. Metode main - Eksekusi dan Output

graph_2011.printGraph_2011();: Menampilkan adjacency list untuk verifikasi struktur.

graph_2011.dfs_2011("A");: Jalankan DFS dari node "A".

graph_2011.bfs_2011("A");: Jalankan BFS dari node "A".

2.2 Pembahasan Tugas Pratikum

```
package pekan9_2511532011;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.*;
import java.util.List;

public class petaHarian_2511532011 extends JFrame {

    private JComboBox<String> comboBoxAwal_2011;
    private JComboBox<String> comboBoxAkhir_2011;
    private JButton btnBfs_2011;
    private JButton btnDfs_2011;
    private JButton btnReset_2011;
    private JLabel lblJalur_2011;
    private JLabel lblNodeDikunjungi_2011;
    private JLabel lblJumlahNode_2011;
    private GraphPanel_2011 panelGrafik_2011;

    private Map<String, Vertex_2011> vertices_2011;
    private Map<String, List<String>> adjacencyList_2011;
    private List<String> visitedNodes_2011;
    private List<String> pathNodes_2011;

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> {
            try {
                petaHarian_2511532011 frame = new petaHarian_2511532011();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    }

    public petaHarian_2511532011() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(583, 700);
        setLocationRelativeTo(null);
        getContentPane().setLayout(null);

        initializeGraph_2011();

        JLabel lblLokasiAwal_2011 = new JLabel("Lokasi Awal:");
        lblLokasiAwal_2011.setBounds(10, 11, 80, 14);
        getContentPane().add(lblLokasiAwal_2011);

        comboBoxAwal_2011 = new JComboBox<>();
        comboBoxAwal_2011.setBounds(90, 7, 150, 22);
        for (String vertex : vertices_2011.keySet()) {
            comboBoxAwal_2011.addItem(vertex);
        }
        getContentPane().add(comboBoxAwal_2011);

        JLabel lblLokasiAkhir_2011 = new JLabel("Lokasi Tujuan:");
        lblLokasiAkhir_2011.setBounds(10, 36, 80, 14);
        getContentPane().add(lblLokasiAkhir_2011);

        comboBoxAkhir_2011 = new JComboBox<>();
        comboBoxAkhir_2011.setBounds(90, 32, 150, 22);
        for (String vertex : vertices_2011.keySet()) {
            comboBoxAkhir_2011.addItem(vertex);
        }
        getContentPane().add(comboBoxAkhir_2011);

        btnBfs_2011 = new JButton("BFS");
        btnBfs_2011.setBounds(260, 7, 80, 23);
        btnBfs_2011.addActionListener(e -> BFS_2011());
        getContentPane().add(btnBfs_2011);

        btnDfs_2011 = new JButton("DFS");
        btnDfs_2011.setBounds(350, 7, 80, 23);
        btnDfs_2011.addActionListener(e -> DFS_2011());
        getContentPane().add(btnDfs_2011);

        btnReset_2011 = new JButton("RESET");
        btnReset_2011.setBounds(440, 7, 80, 23);
        btnReset_2011.addActionListener(e -> resetGraph_2011());
        getContentPane().add(btnReset_2011);

        panelGrafik_2011 = new GraphPanel_2011();
        panelGrafik_2011.setBounds(10, 61, 500, 500);
        getContentPane().add(panelGrafik_2011);

        JLabel lblHasilPencarian_2011 = new JLabel("Hasil Pencarian:");
        lblHasilPencarian_2011.setBounds(10, 563, 150, 14);
        getContentPane().add(lblHasilPencarian_2011);

        lblJalur_2011 = new JLabel("Jalur: -");
    }
}
```

```

        lblJalur_2011.setBounds(10, 577, 780, 14);
        getContentPane().add(lblJalur_2011);

        lblNodeDikunjungi_2011 = new JLabel("Node Dikunjungi: -");
        lblNodeDikunjungi_2011.setBounds(10, 595, 780, 14);
        getContentPane().add(lblNodeDikunjungi_2011);

        lblJumlahNode_2011 = new JLabel("Jumlah Node Dikunjungi: 0");
        lblJumlahNode_2011.setBounds(10, 612, 780, 14);
        getContentPane().add(lblJumlahNode_2011);

        displayGraph_2011();
    }

    private void initializeGraph_2011() {
        vertices_2011 = new HashMap<>();
        adjacencyList_2011 = new HashMap<>();

        addVertex_2011("Kantin DPR", 240, 60);
        addVertex_2011("ATB", 100, 110);
        addVertex_2011("DTI", 380, 110);

        addVertex_2011("Gedung Audit", 240, 190);
        addVertex_2011("FISIP", 100, 280);
        addVertex_2011("Gedung H", 380, 280);

        addVertex_2011("Kos A", 80, 390);
        addVertex_2011("Kos B", 200, 390);
        addVertex_2011("Aciak Mart", 320, 390);
        addVertex_2011("Feb Mart", 440, 390);

        addEdge_2011("Kantin DPR", "ATB");
        addEdge_2011("Kantin DPR", "DTI");
        addEdge_2011("Kantin DPR", "Gedung Audit");
        addEdge_2011("ATB", "DTI");

        addEdge_2011("ATB", "Gedung Audit");
        addEdge_2011("DTI", "Gedung H");
        addEdge_2011("DTI", "FISIP");

        addEdge_2011("Gedung Audit", "Gedung H");
        addEdge_2011("Gedung Audit", "Kos B");

        addEdge_2011("FISIP", "Kos A");
        addEdge_2011("FISIP", "Kos B");

        addEdge_2011("Kos A", "Kos B");
        addEdge_2011("Kos B", "Aciak Mart");
        addEdge_2011("Aciak Mart", "Feb Mart");
        addEdge_2011("Feb Mart", "Gedung H");
        addEdge_2011("Aciak Mart", "Gedung H");
    }

    private void addVertex_2011(String name_2011, int x_2011, int y_2011) {
        vertices_2011.put(name_2011, new Vertex_2011(name_2011, x_2011, y_2011));
        adjacencyList_2011.put(name_2011, new ArrayList<>());
    }

    private void addEdge_2011(String v1_2011, String v2_2011) {
        adjacencyList_2011.get(v1_2011).add(v2_2011);
        adjacencyList_2011.get(v2_2011).add(v1_2011);
    }

    public void BFS_2011() {
        String start_2011 = (String) comboBoxAwal_2011.getSelectedItem();
        String goal_2011 = (String) comboBoxAkhir_2011.getSelectedItem();

        if (start_2011 == null || goal_2011 == null || start_2011.equals(goal_2011)) {
            JOptionPane.showMessageDialog(this, "Pilih lokasi awal dan tujuan yang berbeda!");
            return;
        }

        Queue<String> queue_2011 = new LinkedList<>();
        Set<String> visited_2011 = new LinkedHashSet<>();
        Map<String, String> parent_2011 = new HashMap<>();

        queue_2011.add(start_2011);
        visited_2011.add(start_2011);
        boolean found_2011 = false;

        while (!queue_2011.isEmpty()) {
            String current_2011 = queue_2011.poll();

            if (current_2011.equals(goal_2011)) {
                found_2011 = true;
                break;
            }

            List<String> neighbors_2011 = new ArrayList<>(adjacencyList_2011.get(current_2011));
            Collections.sort(neighbors_2011);

            for (String neighbor_2011 : neighbors_2011) {
                if (!visited_2011.contains(neighbor_2011)) {
                    visited_2011.add(neighbor_2011);
                }
            }
        }
    }

```

```

        parent_2011.put(neighbor_2011, current_2011);
        queue_2011.add(neighbor_2011);
    }
}

if (found_2011) {
    List<String> path_2011 = new ArrayList<>();
    String curr_2011 = goal_2011;
    while (curr_2011 != null) {
        path_2011.add(curr_2011);
        curr_2011 = parent_2011.get(curr_2011);
    }
    Collections.reverse(path_2011);
    visitedNodes_2011 = new ArrayList<>(visited_2011);
    pathNodes_2011 = path_2011;
    displayPath_2011("BFS", path_2011, visitedNodes_2011);
} else {
    JOptionPane.showMessageDialog(this, "Jalur tidak ditemukan!");
}
}

public void DFS_2011() {
    String start_2011 = (String) comboBoxAwal_2011.getSelectedItemAt();
    String goal_2011 = (String) comboBoxAkhir_2011.getSelectedItemAt();

    if (start_2011 == null || goal_2011 == null || start_2011.equals(goal_2011)) {
        JOptionPane.showMessageDialog(this, "Pilih lokasi awal dan tujuan yang berbeda!");
        return;
    }

    Stack<String> stack_2011 = new Stack<>();
    Set<String> visited_2011 = new LinkedHashSet<>();
    Map<String, String> parent_2011 = new HashMap<>();

    stack_2011.push(start_2011);
    visited_2011.add(start_2011);
    boolean found_2011 = false;

    while (!stack_2011.isEmpty()) {
        String current_2011 = stack_2011.pop();

        if (current_2011.equals(goal_2011)) {
            found_2011 = true;
            break;
        }

        List<String> neighbors_2011 = new ArrayList<>(adjacencyList_2011.get(current_2011));
        Collections.sort(neighbors_2011);
        Collections.reverse(neighbors_2011);

        for (String neighbor_2011 : neighbors_2011) {
            if (!visited_2011.contains(neighbor_2011)) {
                visited_2011.add(neighbor_2011);
                parent_2011.put(neighbor_2011, current_2011);
                stack_2011.push(neighbor_2011);
            }
        }
    }

    if (found_2011) {
        List<String> path_2011 = new ArrayList<>();
        String curr_2011 = goal_2011;
        while (curr_2011 != null) {
            path_2011.add(curr_2011);
            curr_2011 = parent_2011.get(curr_2011);
        }
        Collections.reverse(path_2011);
        visitedNodes_2011 = new ArrayList<>(visited_2011);
        pathNodes_2011 = path_2011;
        displayPath_2011("DFS", path_2011, visitedNodes_2011);
    } else {
        JOptionPane.showMessageDialog(this, "Jalur tidak ditemukan!");
    }
}

public void displayGraph_2011() {
    panelGrafik_2011.repaint();
}

public void displayPath_2011(String method_2011, List<String> path_2011, List<String> visited_2011) {
    StringBuilder hasil_2011 = new StringBuilder();
    hasil_2011.append("<html><body style='width: 250px;'>");
    hasil_2011.append("<b>Metode: </b>").append(method_2011).append("<br><br>");
    hasil_2011.append("<b>Jalur Ditemukan:</b><br>");
    hasil_2011.append(String.join(" &arr; ", path_2011)).append("<br><br>");
    hasil_2011.append("<b>Urutan Node Dikunjungi:</b><br>");
    hasil_2011.append(String.join(" ", visited_2011)).append("<br><br>");
    hasil_2011.append("<b>Jumlah Node Dikunjungi:</b> ").append(visited_2011.size());
    hasil_2011.append("</body></html>");

    lblJalur_2011.setText("Jalur: " + String.join(" -> ", path_2011));
    lblNodeDikunjungi_2011.setText("Node Dikunjungi: " + String.join(" ", visited_2011));
    lblJumlahNode_2011.setText("Jumlah Node Dikunjungi: " + visited_2011.size());
}

```

```

        displayGraph_2011();
    }

    public void resetGraph_2011() {
        visitedNodes_2011 = new ArrayList<>();
        pathNodes_2011 = new ArrayList<>();
        lblJalur_2011.setText("Jalur: -");
        lblNodeDikunjungi_2011.setText("Node Dikunjungi: -");
        lblJumlahNode_2011.setText("Jumlah Node Dikunjungi: 0");
        displayGraph_2011();
    }

    private static class Vertex_2011 {
        String name_2011;
        int x_2011, y_2011;
        Color color_2011 = Color.WHITE;

        Vertex_2011(String name_2011, int x_2011, int y_2011) {
            this.name_2011 = name_2011;
            this.x_2011 = x_2011;
            this.y_2011 = y_2011;
        }
    }

    private class GraphPanel_2011 extends JPanel {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2d_2011 = (Graphics2D) g;
            g2d_2011.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

            g2d_2011.setColor(Color.GRAY);
            g2d_2011.setStroke(new BasicStroke(2));
            for (String v1_2011 : adjacencyList_2011.keySet()) {
                Vertex_2011 vertex1_2011 = vertices_2011.get(v1_2011);
                for (String v2_2011 : adjacencyList_2011.get(v1_2011)) {
                    Vertex_2011 vertex2_2011 = vertices_2011.get(v2_2011);
                    g2d_2011.drawLine(vertex1_2011.x_2011, vertex1_2011.y_2011, vertex2_2011.x_2011,
vertex2_2011.y_2011);
                }
            }

            for (Vertex_2011 v_2011 : vertices_2011.values()) {
                if (pathNodes_2011 != null && pathNodes_2011.contains(v_2011.name_2011)) {
                    v_2011.color_2011 = Color.GREEN;
                } else if (visitedNodes_2011 != null && visitedNodes_2011.contains(v_2011.name_2011)) {
                    v_2011.color_2011 = Color.YELLOW;
                } else {
                    v_2011.color_2011 = Color.WHITE;
                }

                g2d_2011.setColor(v_2011.color_2011);
                g2d_2011.fillOval(v_2011.x_2011 - 25, v_2011.y_2011 - 25, 50, 50);
                g2d_2011.setColor(Color.BLACK);
                g2d_2011.setStroke(new BasicStroke(2));
                g2d_2011.drawOval(v_2011.x_2011 - 25, v_2011.y_2011 - 25, 50, 50);

                g2d_2011.setFont(new Font("SansSerif", Font.BOLD, 11));
                FontMetrics fm_2011 = g2d_2011.getFontMetrics();
                int textWidth_2011 = fm_2011.stringWidth(v_2011.name_2011);
                g2d_2011.drawString(v_2011.name_2011, v_2011.x_2011 - (textWidth_2011 / 2), v_2011.y_2011 + 40);
            }
        }
    }
}

```

Deskripsi Peta:

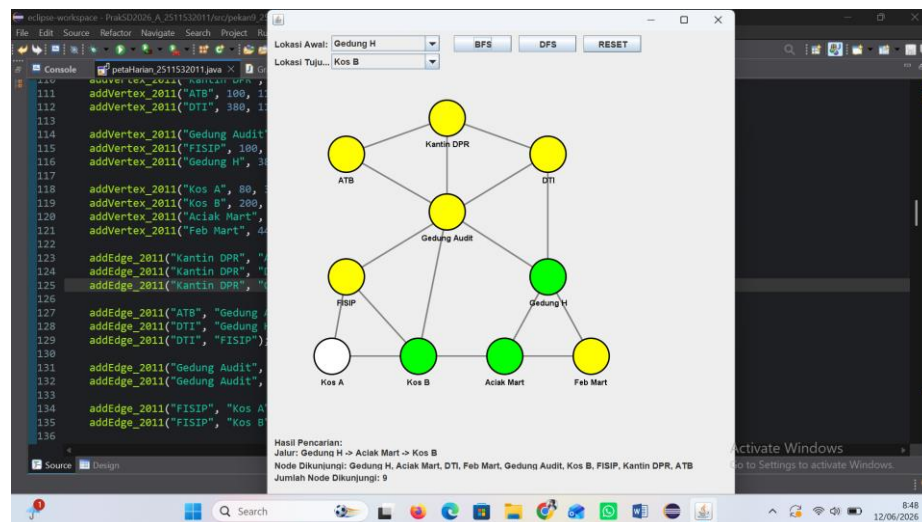
Peta yang saya buat ini merepresentasikan area kampus dan sekitarnya, yang mencakup gedung kuliah, tempat makan, hingga area kos-kosan. Graph ini memiliki 10 simpul yang mewakili lokasi, seperti gedung akademik (DTI, FISIP, ATB, Gedung Audit, dan Gedung H), fasilitas kuliner (Kantin DPR, Aciak Mart, dan Feb Mart), serta tempat tinggal (Kos A dan Kos B). Kesepuluh lokasi ini saling terhubung melalui 16 sisi yang menggambarkan jalur jalan atau akses antar tempat. Peta ini dibuat untuk mensimulasikan mobilitas sehari-hari mahasiswa,

seperti saat mencari rute tercepat untuk pindah kelas, mencari tempat makan, atau kembali ke kos.

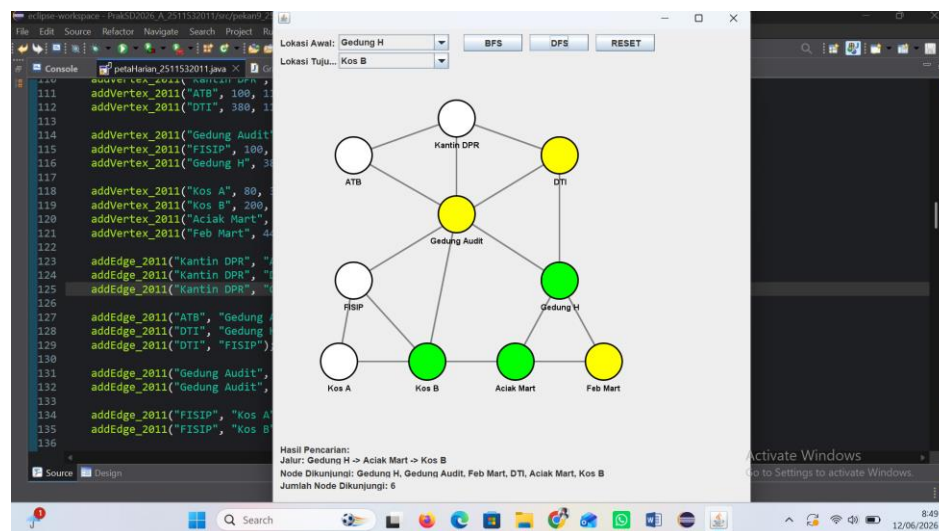
Jumlah Node: 10

Jumlah Sisi: 16

Hasil BFS



Hasil DFS



Kesimpulan Perbandingan DFS dan BFS:

Berdasarkan hasil implementasi algoritma BFS dan DFS pada pencarian jalur dari Gedung H ke Kos B, dapat disimpulkan bahwa kedua algoritma berhasil menemukan jalur yang sama yaitu 'Gedung H → Aciak Mart → Kos B', namun

dengan efisiensi yang berbeda. BFS mengeksplorasi 9 node dengan pola pencarian melebar (level by level), sedangkan DFS hanya mengeksplorasi 6 node dengan pola pencarian mendalam (menelusuri satu cabang sampai akhir). Hal ini menunjukkan bahwa DFS lebih efisien dalam hal jumlah node yang dikunjungi dan penggunaan memori untuk kasus ini, namun BFS tetap memiliki keunggulan dalam menjamin jalur terpendek yang optimal karena memeriksa semua kemungkinan secara sistematis. Dengan demikian, pemilihan algoritma harus disesuaikan dengan kebutuhan: gunakan BFS jika prioritas utama adalah menemukan jalur terpendek yang optimal, sedangkan DFS lebih cocok ketika efisiensi memori dan kecepatan pencarian menjadi prioritas tanpa harus menjamin jalur terpendek.

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan hasil praktikum, dapat disimpulkan bahwa implementasi struktur data non-linear berupa Pohon Biner (Binary Tree) dan Graf telah berhasil dilakukan dengan memanfaatkan konsep rekursi dan manajemen referensi (pointer) yang efektif. Pada struktur Pohon Biner, kelas Node dan Btree berhasil dirancang untuk mengelola hierarki data melalui pointer anak kiri dan kanan, serta mengimplementasikan metode penelusuran (traversal) Preorder, Inorder, dan Postorder yang fundamental untuk berbagai operasi seperti pengurutan data dan duplikasi struktur pohon. Sementara itu, pada struktur Graf, representasi Adjacency List menggunakan kombinasi HashMap dan ArrayList terbukti efisien dalam memetakan hubungan antar simpul, yang kemudian berhasil ditelusuri menggunakan algoritma Depth-First Search (DFS) berbasis rekursi dengan pelacakan HashSet untuk mencegah perulangan siklus, serta Breadth-First Search (BFS) yang memanfaatkan struktur data antrean (Queue) FIFO untuk penelusuran berbasis level secara sistematis. Secara keseluruhan, praktikum ini menegaskan pentingnya pemahaman mendalam mengenai rekursi, enkapsulasi, dan pemilihan struktur data pendukung yang tepat dalam mengoptimalkan efisiensi serta menjamin kebenaran logika algoritma penelusuran pada struktur data yang kompleks.

DAFTAR PUSTAKA

[1]W3schools,“JavaDataStructur”2026[Daring].Tersediapada:https://www.w3schools.com/java/java_data_structures.asp.[Diakses:9-juni-2026]

